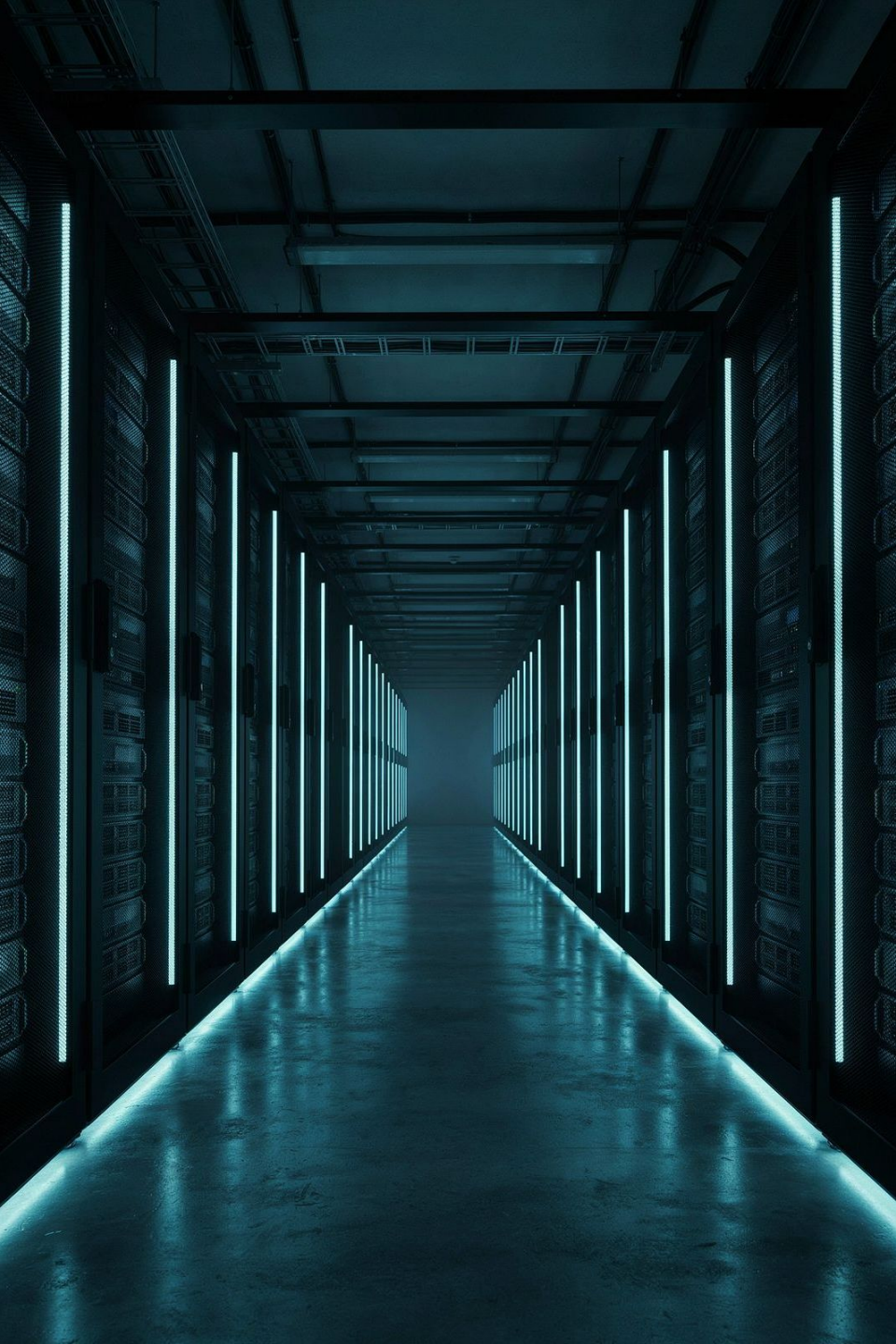




Bases de datos NoSQL

S3.2 · MÓDULO DE BASES DE DATOS

Una guía práctica para entender el ecosistema NoSQL: desde sus fundamentos y tipologías hasta su aplicación real con MongoDB. Aprende cuándo y cómo elegir la solución de almacenamiento adecuada para cada proyecto.

¿Qué veremos hoy?

01

Por qué NoSQL

El origen del movimiento y sus motivaciones reales

02

Tipos de bases de datos NoSQL

Documental, clave-valor, columnar y grafos

03

SQL frente a NoSQL

Comparativa honesta de ventajas y limitaciones

04

MongoDB en profundidad

Documentos, colecciones y operaciones CRUD

05

Cuándo y cómo elegir

Criterios de selección y buenas prácticas

¿Por qué NoSQL?

Los sistemas relacionales nacieron en los años 70 para un mundo de datos estructurados y predecibles. Hoy, las aplicaciones modernas generan **volúmenes masivos de datos heterogéneos** que los modelos clásicos no gestionan bien.



Escala horizontal

Distribuye datos en múltiples nodos sin sacrificar rendimiento



Esquema flexible

Adapta la estructura de datos sin migraciones costosas



Alto rendimiento

Optimizado para lecturas y escrituras de baja latencia



Datos diversos

Almacena JSON, grafos, series temporales y más



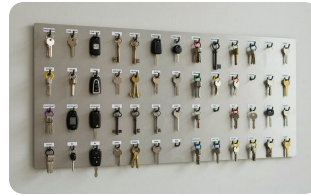
Tipos de bases de datos NoSQL

No existe un único modelo NoSQL. Cada tipo está optimizado para un patrón de acceso concreto. Elegir bien el tipo es tan importante como elegir la tecnología específica.



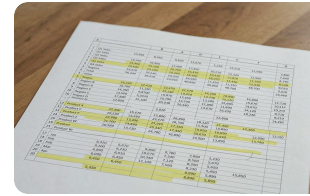
Documental

Almacena datos en documentos JSON o BSON. Ideal para catálogos, perfiles de usuario y contenido variable.



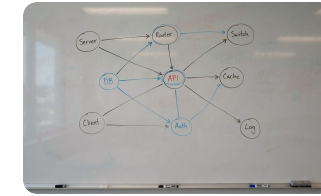
Clave-Valor

Acceso ultrarrápido por clave única. Perfecto para caché, sesiones y configuración distribuida.



Columnar

Organiza datos por columnas en lugar de filas. Excelente para analítica y series temporales a gran escala.



Grafos

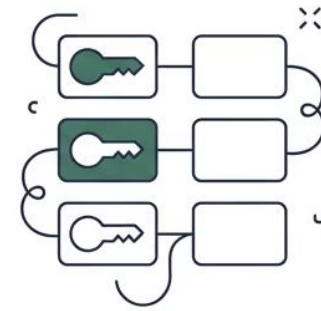
Modela relaciones complejas entre entidades. Ideal para redes sociales, recomendaciones y detección de fraude.

Tipos NoSQL: ejemplos reales



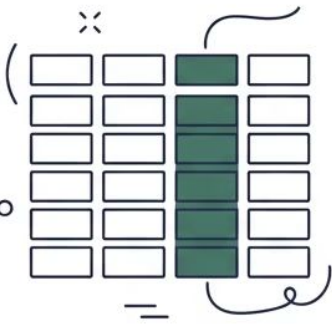
DOCUMENTAL

MongoDB
CouchDB
Firestore



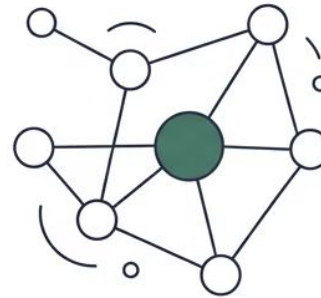
CLAVE-VALOR

Redis
DynamoDB
Memcached



COLUMNAR

Apache
Cassandra
HBase
Google Bigtable



GRAFOS

Neo4j
Amazon
Neptune
ArangoDB

Cada tecnología ha sido diseñada con un caso de uso principal en mente. Conocer sus fortalezas evita costosos rediseños a mitad del proyecto.

SQL frente a NoSQL

No se trata de cuál es mejor, sino de cuál encaja con tu problema. Ambos enfoques tienen su lugar en la arquitectura moderna de datos.

Característica	SQL (Relacional)	NoSQL
Esquema	Rígido, predefinido	Flexible o inexistente
Escalado	Vertical (más hardware)	Horizontal (más nodos)
Transacciones	ACID completo	Eventual consistency (BASE)
Consultas	SQL estándar y JOINS	APIs propias, sin JOINS complejos
Madurez	Décadas de ecosistema	Tecnologías más recientes
Caso ideal	Datos relacionales, banca, ERP	Big Data, tiempo real, IoT

MongoDB: documentos y colecciones

MongoDB es la base de datos documental más popular del mundo. En lugar de tablas y filas, trabaja con **colecciones** (equivalentes a tablas) que contienen **documentos** en formato BSON (Binary JSON).

Conceptos clave

Base de datos

Contenedor de colecciones, equivale a un schema en SQL

Colección

Agrupación de documentos, equivale a una tabla relacional

Documento

Objeto JSON con pares clave-valor, equivale a una fila

_id

Identificador único obligatorio generado automáticamente

Ejemplo de documento

```
{
  "_id": ObjectId("64a1f2..."),
  "nombre": "Ana García",
  "edad": 28,
  "email": "ana@ejemplo.com",
  "habilidades": [
    "Python", "MongoDB"
  ],
  "direccion": {
    "ciudad": "Madrid",
    "cp": "28001"
  }
}
```



CRUD en MongoDB

Las cuatro operaciones fundamentales sobre datos — **Create, Read, Update, Delete** — se ejecutan en MongoDB a través de métodos intuitivos sobre colecciones. A continuación, los ejemplos más habituales.

1

Create — Insertar

```
db.usuarios.insertOne({
  nombre: "Luis", edad: 32
})
```

Usa `insertMany()` para insertar varios documentos a la vez.

2

Read — Consultar

```
db.usuarios.find(
  { edad: { $gt: 25 } }
)
```

Filtra con operadores como `$gt`, `$lt`, `$eq`, `$in` y más.

3

Update — Actualizar

```
db.usuarios.updateOne(
  { nombre: "Luis" },
  { $set: { edad: 33 } }
)
```

Usa `updateMany()` para modificar múltiples documentos simultáneamente.

4

Delete — Eliminar

```
db.usuarios.deleteOne(
  { nombre: "Luis" }
)
```

Usa `deleteMany()` con precaución; sin filtro elimina toda la colección.

CRUD: operadores de consulta más usados

Operador	Significado	Ejemplo
\$eq	Igual a	{edad: {\$eq: 30}}
\$gt / \$lt	Mayor / Menor	{edad: {\$gt: 18}}
\$in	Dentro de un array	{rol: {\$in: ["admin"]}}
\$and / \$or	Lógica combinada	{\$and: [{...},{...}]}
\$exists	Campo existe	{email: {\$exists: true}}
\$set	Modifica campo	{\$set: {activo: true}}

Consulta con proyección

```
db.usuarios.find(  
  { activo: true },  
  { nombre: 1, email: 1, _id: 0 }  
)
```

El segundo argumento de `find()` es la **proyección**: indica qué campos devolver (1) u ocultar (0). Útil para reducir el tamaño de las respuestas.

 Nunca mezcles inclusiones y exclusiones en la misma proyección (salvo con `_id`).

¿Cuándo elegir cada tipo?

La elección del tipo de base de datos debe basarse en el **patrón de acceso** a los datos, no en la popularidad de la tecnología. Estas preguntas guían la decisión:

¿Tus datos tienen estructura fija y relaciones complejas?

→ **SQL relacional**. Las transacciones ACID y los JOINS son tu aliado cuando la integridad es crítica (banca, ERP, reservas).

¿Cada entidad puede tener campos distintos o anidados?

→ **Documental (MongoDB)**. Perfecta para catálogos de productos, CMS, perfiles de usuario y APIs REST.

¿Necesitas caché, sesiones o configuración distribuida?

→ **Clave-Valor (Redis)**. Latencia de microsegundos y millones de operaciones por segundo.

¿Analizas terabytes de eventos o métricas en el tiempo?

→ **Columnar (Cassandra / Bigtable)**. Diseñada para escritura masiva y consultas analíticas eficientes.

¿Las relaciones entre entidades son tan importantes como los datos?

→ **Grafos (Neo4j)**. Redes sociales, motores de recomendación y análisis de fraude son su hábitat natural.

Buenas prácticas en NoSQL

Trabajar con bases de datos NoSQL requiere cambiar el chip respecto al diseño relacional. Estas prácticas evitan los errores más frecuentes en producción.

Diseña según el acceso

Modela tus documentos en función de cómo los vas a consultar, no de cómo se relacionan lógicamente. Desnormaliza cuando tenga sentido.

Usa índices con criterio

Crea índices sobre los campos que filtras con frecuencia. Demasiados índices ralentizan las escrituras y consumen memoria.

Valida los datos

La flexibilidad del esquema no significa ausencia de reglas. Usa validación de esquema (JSON Schema en MongoDB) para garantizar calidad.

Monitoriza el rendimiento

Analiza el plan de ejecución de tus consultas con `explain()`. Identifica *collection scans* y optimízalos con índices adecuados.

Planifica copias de seguridad

NoSQL no es sinónimo de invulnerable. Establece políticas de backup y prueba regularmente la restauración de datos.

Tamaño de documentos

En MongoDB, el límite es 16 MB por documento. Evita documentos excesivamente grandes; usa referencias cuando los arrays crecen sin control.



Conclusiones y próximos pasos



NoSQL no reemplaza SQL

Son herramientas complementarias. La clave está en elegir la correcta según el problema.



Cada tipo tiene su nicho

Documental, clave-valor, columnar y grafos cubren casos de uso radicalmente distintos.



MongoDB es un excelente punto de partida

Su API intuitiva y su flexibilidad la convierten en la puerta de entrada ideal al mundo NoSQL.



Practica con CRUD

Dominar las cuatro operaciones básicas es el primer paso antes de abordar aggregations y sharding.



Próximo paso recomendado: instala MongoDB Community Edition o usa MongoDB Atlas (gratuito) y replica los ejemplos CRUD de esta sesión con un dataset real.